

ATHLETE-SHIELD AI: INTELLIGENT ATHLETE INJURY RISK PREDICTION & PERFORMANCE MONITORING

Git & Docker Technical Assessment

Submitted partial fulfilment for the Course of

CSA1012 Software Engineering

to receive the award of the degree of

INFORMATION TECHNOLOGY

Submitted by

VENNAMPALLI BOOPATHY KHANISHK (192524311)

Under the Supervision of ANITADAVAMANI K

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

August-2026

1. Problem Analysis

1.1 Problem Statement Overview

Sports organizations and training academies increasingly rely on data-driven methods to safeguard athlete health and optimize performance. Despite the growing use of wearable sensors and fitness trackers, most existing systems record raw activity data without translating it into meaningful, early warning signals for injury risk. Coaches and physiotherapists are often left to make training-load decisions based on intuition rather than continuous, quantified insight into how an athlete's body is responding to training stress.

Athlete-Shield AI is proposed to address this gap. It is an intelligent platform that continuously monitors an athlete's training load, physiological signals, and biomechanical stress indicators gathered from wearable and sensor devices, and applies machine learning to compute a real-time injury risk score. The platform also tracks performance trends over time, enabling coaches and sports medicine staff to make informed decisions that reduce injury incidence and support sustained athletic performance.

1.2 Project Objectives

- Develop an intelligent platform for continuous athlete health and performance monitoring.
- Predict individualized injury risk scores using machine learning models.
- Track training load, workload ratios, and recovery indicators over time.
- Integrate wearable, GPS, and IMU sensor data for real-time monitoring.
- Visualize performance and injury-risk trends through interactive dashboards.
- Alert coaches and physiotherapists when an athlete's injury risk becomes elevated.
- Ensure secure storage and management of sensitive athlete health data.
- Support long-term athlete development through historical performance analytics.

1.3 Functional Requirements

- Athlete registration and profile management (age, sport, position, injury history).
- Training load logging module (session type, duration, intensity, perceived exertion).
- Sensor data ingestion module for GPS distance/speed, heart rate, and IMU acceleration data.
- AI-based injury risk prediction engine that computes a risk score from training and physiological data.

- Coach/physiotherapist dashboard displaying workload ratios, risk trends, and performance charts.
- Alert and notification module for athletes flagged as high injury risk.
- Secure authentication and encrypted storage of athlete health and performance records.
- Historical reporting module for long-term performance and injury trend analysis.

1.4 Expected Outcomes

- A fully functional athlete injury-risk prediction and performance monitoring platform.
- Individualized, data-driven injury risk scores for every registered athlete.
- Real-time monitoring of training load and biomechanical stress through sensor integration.
- Early identification of high-risk athletes before injuries occur.
- Improved, evidence-based decision-making by coaches and sports medicine staff.
- Accurate visualization of performance and risk trends over the training season.
- Secure and compliant management of sensitive athlete health data.
- Reduced injury incidence and improved overall athletic performance and wellbeing.

2. Project Structure Design

2.1 Repository Folder Structure

The project repository is organized into independent, service-based folders so that each functional unit of Athlete-Shield AI can be developed, tested, and containerized separately.

```
AthleteShieldAI/
├── frontend/           # React dashboard for coaches, physios & athletes
├── backend/            # Node.js + Express REST API
├── ai-service/         # Python service for injury risk prediction
├── sensor-service/     # Wearable / GPS / IMU data ingestion & sync
├── alert-service/      # High-risk athlete alerts & notifications
├── database/           # DB init scripts and schema/migrations
├── docs/               # Project documentation and diagrams
├── docker/             # Service-specific Dockerfiles and configs
├── .gitignore
├── README.md
├── Dockerfile
└── docker-compose.yml
```

2.2 Purpose of Each Major Folder and File

- frontend/ – Contains the coach/physiotherapist/athlete dashboard where users view training load, injury-risk scores, and performance charts.
- backend/ – Hosts REST APIs that handle authentication, athlete profiles, training-load logs, and core business logic.
- ai-service/ – Runs the machine learning model that computes injury-risk scores from training and physiological data.
- sensor-service/ – Manages synchronization with wearable, GPS, and IMU sensor APIs to pull heart rate, distance, speed, and acceleration data.
- alert-service/ – Detects high injury-risk conditions and dispatches alerts to coaches and physiotherapists.
- database/ – Stores schema definitions and seed/migration scripts for athlete, training, and sensor data.
- docs/ – Holds architecture diagrams, API documentation, and assignment reports.
- docker/ – Contains additional Dockerfiles and environment configuration files per service.
- .gitignore – Excludes node_modules, environment files, logs, and build artifacts from version control.
- README.md – Describes the project, setup instructions, and usage guidelines.
- Dockerfile – Defines how the primary application image is built.
- docker-compose.yml – Orchestrates all services, networks, and volumes together.

3. Git Repository Initialization

3.1 Steps for Repository Initialization

The following steps were used to initialize the Git repository and publish it to a remote host (GitHub):

```
git init
git add .
git commit -m "Initial project setup"
git branch -M main
git remote add origin <repository-url>
git push -u origin main
```

git init creates a new local repository and starts tracking the project directory. git add . stages all project files for commit. git commit -m records the staged changes with a descriptive message. git branch -M main renames the default branch to main. git remote add origin links the local repository

to the remote GitHub repository, and `git push -u origin main` uploads the initial commit and sets the upstream tracking branch.

3.2 Purpose of Essential Git Files

- `.git/` – Hidden directory automatically created by `git init`; stores all version history, commit objects, and configuration.
- `.gitignore` – Prevents unnecessary or sensitive files (`node_modules`, `.env`, log files, model checkpoints) from being tracked.
- `README.md` – Provides an overview of Athlete-Shield AI, setup steps, and API usage for new contributors.
- `package.json` / `requirements.txt` – Define dependencies for the Node.js and Python services respectively, ensuring reproducible environments.

4. Git Branching Strategy

4.1 Branching Model Design

A feature-based branching strategy (similar to Git Flow) was adopted for the Athlete-Shield AI project. The model separates stable, in-progress, and experimental code so that multiple modules — athlete profiles, training-load tracking, injury-risk prediction, sensor integration, and alert notifications — can be developed independently.

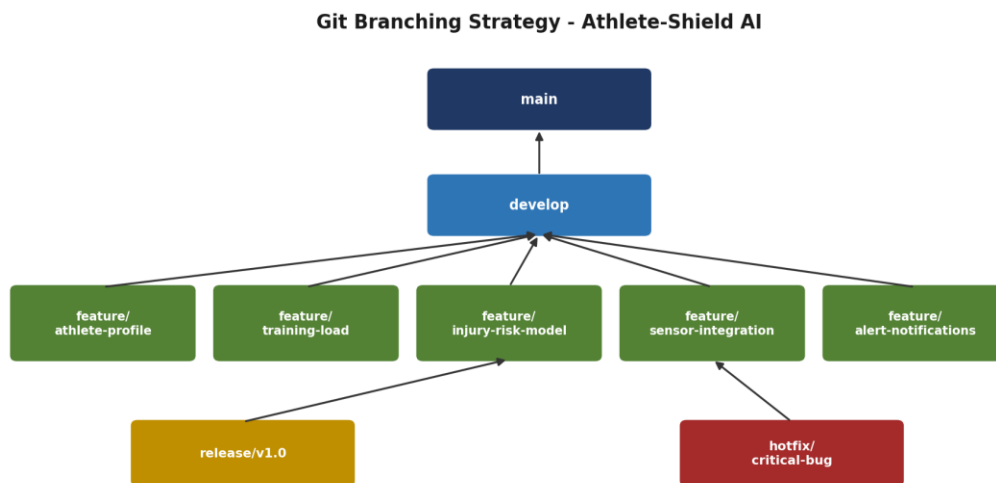


Figure 1: Git Branching Strategy - Athlete-Shield AI

4.2 Justification of the Branching Model

- **main** – Always holds production-ready, stable code that is deployed to users.
- **develop** – Integration branch where completed features are merged and tested together before release.
- **feature/*** – Isolated branches (e.g., `feature/training-load`, `feature/injury-risk-model`) allow each module to be built without affecting others.
- **release/*** – Used to prepare a version for production, including final testing and documentation.
- **hotfix/*** – Allows urgent production bugs (e.g., an incorrect training-load calculation) to be fixed quickly without waiting for the next release cycle.

This model is appropriate for Athlete-Shield AI because it involves several loosely coupled services (injury-risk prediction, sensor synchronization, alert notifications) that benefit from parallel, isolated development while keeping the main branch always deployable.

5. Version Control Workflow

5.1 Commits, Branch Creation and Merging

```
# create and switch to a feature branch
git checkout -b feature/injury-risk-model

# stage and commit meaningful changes
git add ai-service/risk_model.py
git commit -m "Add ML-based injury risk scoring logic"

# push the feature branch
git push origin feature/injury-risk-model

# merge into develop after review
git checkout develop
git merge feature/injury-risk-model
```

5.2 Conflict Resolution

When two developers modified the same section of the `docker-compose.yml` file (one adding the `sensor-service` and another adding the `alert-service`), Git flagged a merge conflict. The conflict markers (`<<<<<<<`, `=====`, `>>>>>>>`) were reviewed manually, both service definitions were retained, and the file was re-staged and committed to complete the merge:

```
git status
# resolve conflicting lines in docker-compose.yml
git add docker-compose.yml
git commit -m "Resolve merge conflict in docker-compose service definitions"
```

5.3 Repository Synchronization

```
git pull origin develop # fetch and merge remote changes
git push origin develop # publish local commits to the remote
```

Each Git operation serves a distinct purpose: checkout -b creates isolated workspaces for new features, add/commit records incremental and meaningful progress, push/pull keep the local and remote repositories synchronized, and merge integrates completed work back into shared branches.

6. Commit History Analysis

6.1 Sample Commit History

Commit Hash	Commit Message	Branch
a1c3f9e	Initial project setup	main
b7d21aa	Add athlete profile and authentication module	feature/athlete-profile
c93ef01	Implement training load and session tracking	feature/training-load
d45bc72	Add ML-based injury risk scoring logic	feature/injury-risk-model
e88a10f	Integrate wearable and GPS sensor data sync	feature/sensor-integration
f12d9b3	Add high-risk athlete alert scheduler	feature/alert-notifications
019cfa4	Resolve merge conflict in docker-compose service definitions	develop
12e7b56	Add Dockerfile and docker-compose configuration	develop
2a4f8dd	Prepare release v1.0 with documentation updates	release/v1.0

6.2 Commit Message Practices

- Each commit message uses the imperative mood (e.g., "Add", "Implement", "Resolve") to clearly describe the change.
- Messages are short, specific, and scoped to a single logical change, which makes the history easy to read with git log --oneline.
- Grouping commits by feature (profile, training load, risk prediction, sensor sync, alerts) allows any module to be traced or reverted independently.
- Meaningful commit history simplifies debugging, code review, and onboarding of new developers to the Athlete-Shield AI project.

7. Docker Environment Design

7.1 Why Docker is Suitable

Athlete-Shield AI consists of multiple components — a JavaScript frontend, a Node.js backend, a Python-based injury-risk prediction engine, a sensor-sync service, an alert service, and a database — each with different runtimes and dependencies. Docker packages every component into an isolated, reproducible container, eliminating "it works on my machine" issues and simplifying deployment across development, testing, and production environments.

7.2 Application Components Requiring Containerization

- Frontend – Serves the coach/physiotherapist/athlete dashboard interface.
- Backend API – Handles authentication, athlete profile, and training-load endpoints.
- AI Injury-Risk Prediction Service – Runs the Python-based machine learning model.
- Sensor Sync Service – Connects to third-party wearable, GPS, and IMU device APIs.
- Alert Service – Sends scheduled and event-driven high-risk notifications.
- Database – Stores athlete profiles, training logs, sensor data, and risk-score history.

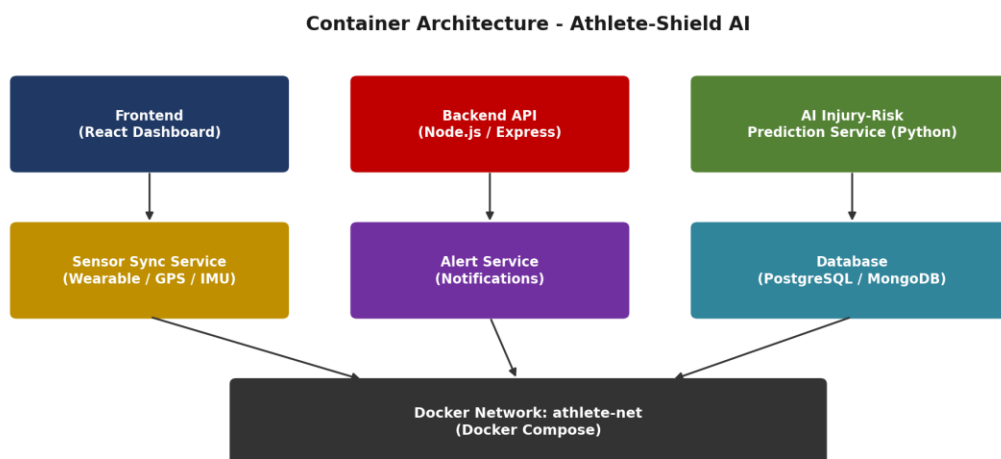


Figure 2: Container Architecture - Athlete-Shield AI

8. Dockerfile Development

8.1 Dockerfile for the Backend Service

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
```



```
EXPOSE 5000
CMD ["npm", "start"]
```

8.2 Dockerfile for the AI Injury-Risk Prediction Service

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 6000
CMD ["python", "app.py"]
```

8.3 Instruction-by-Instruction Explanation

- FROM – Selects the base image (Node.js Alpine for the backend, slim Python for the AI service) to keep image size minimal.
- WORKDIR – Sets /app as the working directory inside the container for all subsequent instructions.
- COPY package*.json ./ / COPY requirements.txt . – Copies only dependency manifests first so Docker can cache the install layer.
- RUN npm install / RUN pip install – Installs the required dependencies inside the image.
- COPY . . – Copies the remaining application source code into the container.
- EXPOSE – Documents the port the service listens on (5000 for backend, 6000 for AI service).
- CMD – Specifies the default command executed when the container starts.

9. Docker Compose Design

9.1 docker-compose.yml Configuration

```
version: "3.9"
services:
  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on:
      - backend
    networks:
      - athlete-net
```

```
backend:
  build: ./backend
  ports:
    - "5000:5000"
  env_file: ./backend/.env
  depends_on:
    - database
    - ai-service
  networks:
    - athlete-net

ai-service:
  build: ./ai-service
  ports:
    - "6000:6000"
  networks:
    - athlete-net

sensor-service:
  build: ./sensor-service
  env_file: ./sensor-service/.env
  depends_on:
    - backend
  networks:
    - athlete-net

alert-service:
  build: ./alert-service
  depends_on:
    - backend
  networks:
    - athlete-net

database:
  image: postgres:16
  volumes:
    - db-data:/var/lib/postgresql/data
  ports:
    - "5432:5432"
  networks:
    - athlete-net

volumes:
  db-data:

networks:
  athlete-net:
    driver: bridge
```

9.2 Services, Networking, Volumes, Environment Variables and Dependencies

- Services – Six services are defined: frontend, backend, ai-service, sensor-service, alert-service, and database, each built from its own Dockerfile or a public image (postgres:16).
- Networking – All services share a custom bridge network (athlete-net) so they can communicate using service names instead of IP addresses (e.g., backend reaches the database at database:5432).
- Volumes – The db-data named volume persists PostgreSQL data across container restarts, preventing data loss when containers are recreated.
- Environment Variables – Sensitive configuration (database URIs, sensor API keys, JWT secrets) is supplied through env_file rather than being hard-coded into the images.
- Dependencies – depends_on ensures the backend starts only after the database and AI service are available, and dependent services (sensor-service, alert-service) start after the backend.

10. Container Deployment and Testing

10.1 Deployment Commands

```
docker compose build    # builds images for all services
docker compose up -d   # starts all containers in detached mode
docker ps              # lists running containers and their status
```

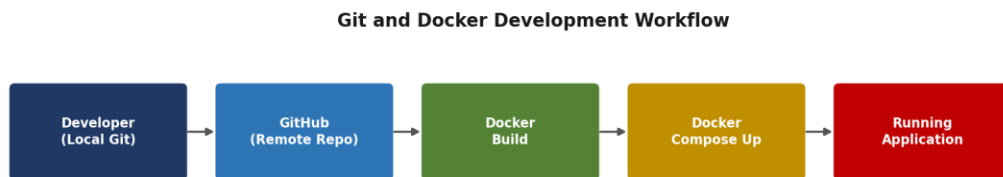


Figure 3: Git and Docker Development Workflow

10.2 Testing Procedure and Evidence

Test Case	Procedure	Expected Result	Status
Container startup	Run docker compose up -d and docker ps	All six containers show status 'Up'	Passed
Frontend load	Open http://localhost:3000 in browser	Dashboard loads with login screen	Passed
Backend connectivity	curl http://localhost:5000/api/health	Returns HTTP 200 with status OK	Passed
Database availability	docker exec into database, run psql	Database accepts connections	Passed

Test Case	Procedure	Expected Result	Status
Injury risk prediction	POST sample athlete data to /api/predict-risk	Returns computed injury risk score	Passed
Sensor sync	Trigger sensor-service sync endpoint	Heart rate and GPS data updated in DB	Passed
Alert dispatch	Trigger scheduled high-risk alert job	Alert logged/sent for high-risk athlete	Passed

Successful execution of all containers, along with verified API responses and inter-service communication, confirms that Athlete-Shield AI runs correctly in a consistent, containerized deployment environment.

11. Benefits of Git and Docker

11.1 Collaboration and Version Management

Git enables multiple developers to work simultaneously on separate modules — athlete profiles, training-load tracking, injury-risk prediction, sensor integration, and alert notifications — without overwriting each other's work. Branching and pull requests provide a structured review process, while the commit history offers a complete, traceable record of how Athlete-Shield AI evolved.

11.2 Portability, Deployment, Scalability and Maintainability

- Portability – Docker containers run identically on a developer's laptop, a testing server, or a cloud platform, eliminating environment inconsistencies.
- Deployment – docker compose up automates the startup of all six services with a single command, reducing manual configuration errors.
- Scalability – Individual services (e.g., ai-service) can be scaled independently using docker compose up --scale as the number of monitored athletes grows.
- Maintainability – Isolated services and a clear branching model make it easier to update, test, and patch individual modules without affecting the entire system.

12. Challenges and Improvements

12.1 Challenges Encountered

- Merge conflicts occurred when multiple developers edited shared configuration files such as docker-compose.yml.
- Coordinating five interdependent services (backend, AI, sensor sync, alerts, database) required careful use of depends_on and networking.

- Managing environment variables and API keys for third-party wearable and GPS integrations securely across containers.
- Ensuring the injury-risk prediction model produced consistent results when redeployed inside a container.
- Keeping feature branches up to date with develop to avoid large, difficult merges.

12.2 Suggested Improvements and Future Enhancements

- Introduce CI/CD pipelines (e.g., GitHub Actions) to automatically build, test, and deploy containers on every push.
- Adopt container orchestration (Kubernetes) for automated scaling and self-healing of services in production.
- Expand wearable integrations to support more devices (Catapult, Polar, Garmin) with standardized data adapters.
- Improve the injury-risk prediction engine using richer models trained on larger, anonymized biomechanical and training-load datasets.
- Add centralized logging and monitoring (e.g., Prometheus and Grafana) for all containerized services.
- Implement automated integration tests that run inside containers as part of the deployment pipeline.